

Multi-Level Paging

32-bit Logical Address: Concept, Translation and Worked Examples

Fundamentals of Operating System Design (CEUC301)

B.Tech. Sem-V

Learning Objectives

- Understand why a single-level page table can become very large.
- Understand the concept of multi-level page tables.
- Divide a 32-bit logical address into page-table indices and offset.
- Translate a logical address using a two-level page table.
- Calculate page-table size and memory references.

Paging: Quick Revision

Basic idea

Logical memory is divided into fixed-size **pages**; physical memory is divided into fixed-size **frames**.

$$\text{Logical Address} = \text{Page Number} + \text{Offset}$$

Page Number $\longrightarrow$ Page Table $\longrightarrow$ Frame Number

$$\text{Physical Address} = \text{Frame Number} + \text{Same Offset}$$

Why Do We Need Multi-Level Paging?

Assume:

- Logical address = 32 bits
- Page size = 4 KB = 2^{12} bytes
- PTE size = 4 bytes

Offset bits:

12

Page-number bits:

$$32 - 12 = 20$$

Number of pages:

$$2^{20} = 1,048,576$$

Single-level page-table size:

$$2^{20} \times 4 = 2^{22} \text{ bytes} = 4 \text{ MB}$$

Problem

A complete 4 MB page table may be wasteful when only a small portion of the virtual address space is used.

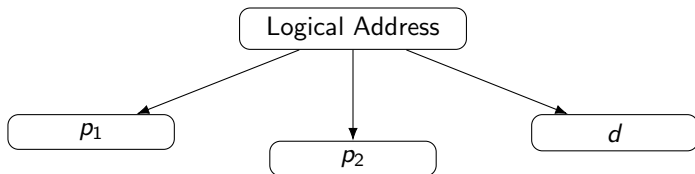
Basic Idea of Multi-Level Paging

Instead of one large page table, divide it into levels.

For two-level paging:

$$\text{Logical Address} = p_1 \mid p_2 \mid d$$

- p_1 : Level-1 page-table index
- p_2 : Level-2 page-table index
- d : page offset



Why Does It Save Memory?

- The Level-1 table contains references to Level-2 tables.
- A Level-2 table is created only for a required region of the address space.
- Unused regions do not require their complete lower-level page tables.

Key idea

Allocate lower-level page tables only where they are needed.

Trade-off

More levels can reduce page-table memory but increase address-translation overhead when the TLB does not contain the translation.

32-bit Example: Given

Consider:

- 32-bit logical addresses
- Page size = 4 KB
- PTE = 4 bytes
- Two-level paging
- Each page table occupies one 4 KB page

A 4 KB page table contains:

$$\frac{4096}{4} = 1024 = 2^{10} \text{ entries}$$

Therefore:

10 bits are needed to index each level

Offset:

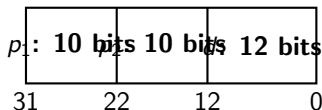
12 bits

32-bit Logical Address Format

$$32 = 10 + 10 + 12$$

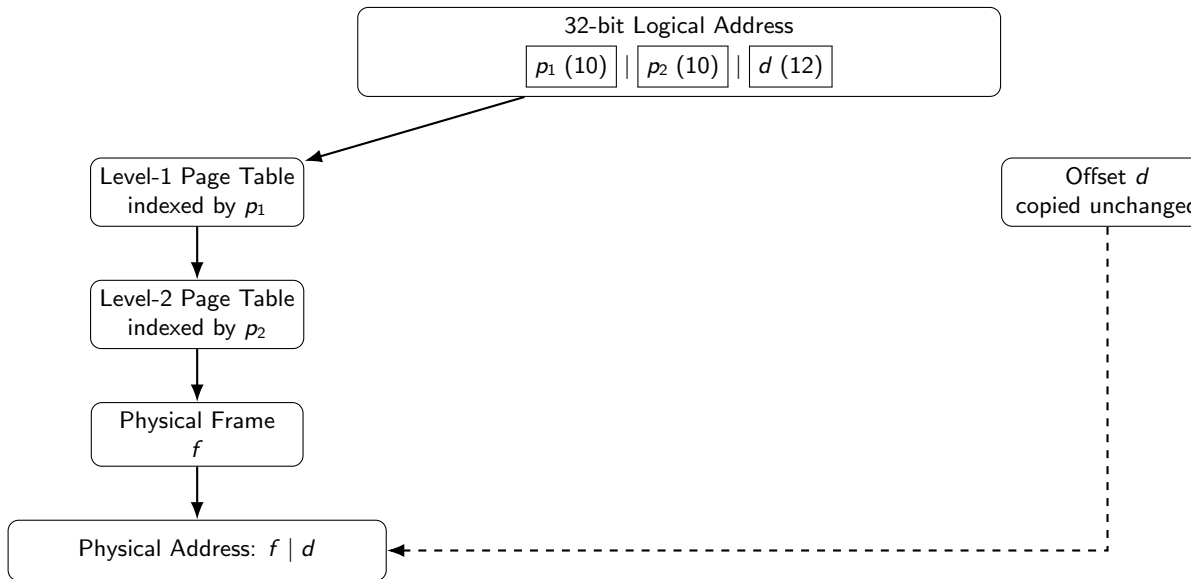
Therefore:

$$\text{Logical Address} = p_1(10) \mid p_2(10) \mid d(12)$$



$$31 \dots 22 \mid 21 \dots 12 \mid 11 \dots 0$$

Two-Level Paging: Address Translation



Worked Example: Logical Address 0x12345678

Given:

0x12345678

Using the 10|10|12 split:

$$0x12345678 = 0001001000\ 1101000101\ 011001111000$$

Hence:

$$p_1 = 72$$

$$p_2 = 837$$

$$d = 1656$$

So the address components are:

$$(p_1, p_2, d) = (72, 837, 1656)$$

Worked Example: Translation

Suppose the Level-2 page-table entry for:

$$p_2 = 837$$

contains:

$$f = 0xABCDE$$

The offset remains:

$$d = 0x678$$

Therefore:

$$\text{Physical Address} = (0xABCDE \ll 12) + 0x678$$

$$= 0xABCDE000 + 0x678$$

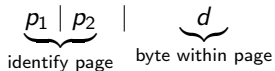
$$\text{Physical Address} = 0xABCDE678$$

Remember

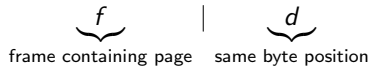
Only the page/frame portion changes. The offset is copied unchanged.

Why Is the Offset Unchanged?

Logical address:



Physical address:



The page table performs:

Virtual Page $\longrightarrow$ Physical Frame

It does **not** change the byte position inside the page.

Memory Accesses

For two-level paging, with a TLB miss:

- 1 Access Level-1 page table.
- 2 Access Level-2 page table.
- 3 Access the required memory word.

Thus:

3 memory accesses

With a TLB hit:

1 memory access

For an n -level page table, a TLB miss requires approximately:

$n + 1$ memory accesses

Page-Table Size Comparison

Each page table:

$$1024 \times 4 = 4096 \text{ bytes} = \boxed{4 \text{ KB}}$$

If only one Level-2 table is required:

$$4 \text{ KB} + 4 \text{ KB} = \boxed{8 \text{ KB}}$$

Single-level page table:

$$\boxed{4 \text{ MB}}$$

Important

The 8 KB figure is for this sparse example. Multi-level paging does not automatically mean the page tables always consume only 8 KB; the actual amount depends on how much of the virtual address space requires lower-level tables.

General Formula

Let:

- Logical address size = m bits
- Page size = 2^d bytes
- PTE size = 2^e bytes

Then:

$$\text{Offset bits} = d$$

Entries in one page-table page:

$$2^{d-e}$$

Index bits per level, when each level occupies one page:

$$d - e$$

For the example:

$$m = 32, \quad d = 12, \quad e = 2$$

$$10 + 10 + 12 = 32$$

Exam-Solving Procedure

- 1 Convert page size to a power of two.
- 2 Offset bits = $\log_2(\text{page size})$.
- 3 Page-number bits = logical-address bits – offset bits.
- 4 Find entries per page-table page:
$$\frac{\text{page size}}{\text{PTE size}}$$
- 5 Convert entries into index bits.
- 6 Split the logical address into level indices and offset.
- 7 Look up the final frame number.
- 8 Combine frame number with the unchanged offset.

Practice Problem

A system has:

- 32-bit logical addresses
- 4 KB pages
- 4-byte PTEs
- Two-level paging

For logical address:

0xCAFEBAE

Find:

1. Offset bits.
2. Bits used by each page-table index.
3. p_1 , p_2 , and offset.
4. If the final frame number is 0x54321, find the physical address.

Practice Problem: Solution

Since:

$$4 \text{ KB} = 2^{12}$$

$$d = 12$$

Each page table:

$$4096/4 = 1024 = 2^{10}$$

entries, so:

$$p_1 = 10, \quad p_2 = 10$$

Binary split:

$$0\text{xCAFEBABE} = 1100101011 \ 1110101010 \ 101111101110$$

Therefore:

$$p_1 = 811, \quad p_2 = 938, \quad d = 0\text{xEBE} = 3774$$

Given:

$$f = 0\text{x54321}$$

$$\text{Physical Address} = 0\text{x54321EBE}$$

Key Takeaways

- Multi-level paging divides a large page table into smaller levels.
- It is especially useful for sparse virtual address spaces.
- For 32-bit addresses and 4 KB pages:

offset = 12 bits

- With 4-byte PTEs, each page table has 1024 entries.
- A two-level format is:

10		10		12
----	--	----	--	----

- The offset remains unchanged during translation.
- A TLB can avoid page-table memory accesses on a hit.

Core Idea

Page-table levels locate the frame; the offset locates the byte.